

LC 133 Clone Graph

Flexport

Nutanix

Amazon

Problem Statement

we are given a reference of a node in a connected undirected graph, need to return a deep copy (clone) of the graph

Requirements

we must create → **NEW nodes**

- same values
- same connections

i.e., new memory, new nodes, can't reuse

The Challenge

we can't simply do DFS bcz

```
1 → 2 (we create 1' ↔ 2')  
1 is seen  
now 2' has a neighbor 1' not yet seen so 1'' ↔ 2'' (infinity recursion)
```

The Solution

so whenever we visit a old node, must ask ourself if already created its clone

Yes → return that clone

No → create one

we need a hashmap for Old node → New Node i.e. 1 → 1'

Implementation

C++ Solution

```
/*
// Definition for a Node.
class Node {
public:
    int val;
    vector<Node*> neighbors;
    Node() {
        val = 0;
        neighbors = vector<Node*>();
    }
    Node(int _val) {
        val = _val;
        neighbors = vector<Node*>();
    }
    Node(int _val, vector<Node*> _neighbors) {
        val = _val;
        neighbors = _neighbors;
    }
};
*/

class Solution {
public:
    unordered_map<Node*, Node*> mp;

    Node* dfs(Node* curr) {
        if (curr == NULL) {
            return NULL;
        }

        // agar already clone exist krta
        if (mp.find(curr) != mp.end()) {
            return mp[curr];
        }

        // agar already clone exist nhi krta to bana lenge
        // value copy
        Node* clone = new Node(curr->val);
        mp[curr] = clone;

        // connections copy
        for (auto &it : curr->neighbors) {
            clone->neighbors.push_back(dfs(it));
        }

        return clone;
    }

    Node* cloneGraph(Node* node) {
        return dfs(node);
    }
};
```